

IEOR 4404: Simulation

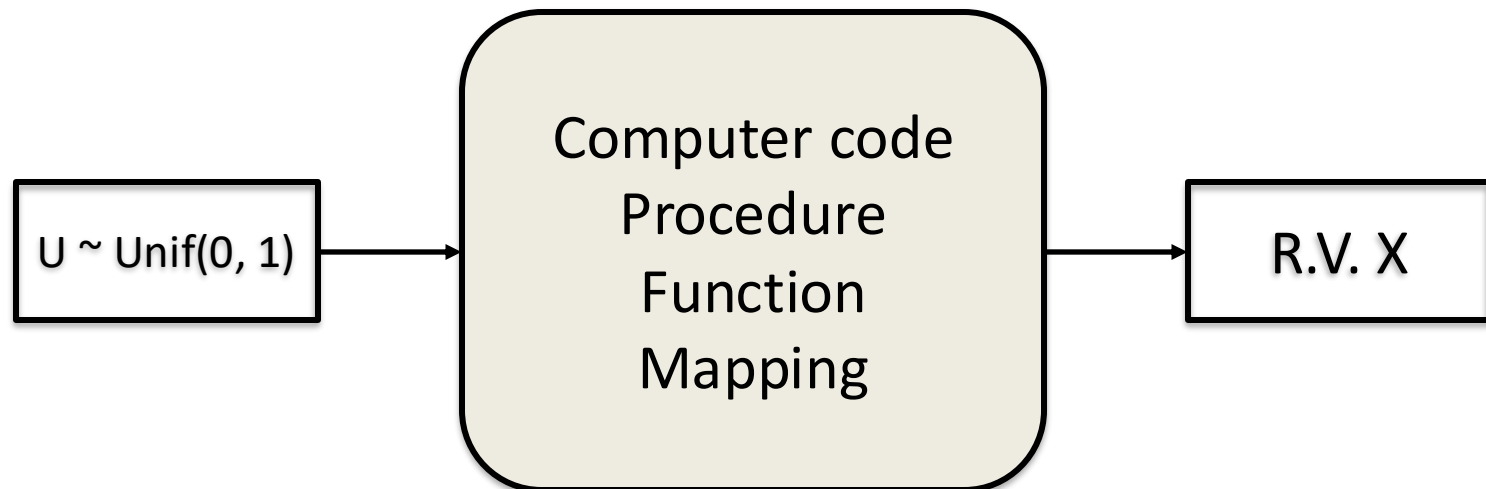
Lecture 3: Random Variable Generation

What is Simulation

- A computational tool for stochastic models with no closed-form
- Requires, as a first task, generation of random variables

A High-Level View of RV Simulation

- What does it mean to generate a random variable?
- In computing languages, $\text{Unif}(0, 1)$ is often taken as the basis for generating other random variables; e.g., $\text{RAND}()$ in Excel; rand in Matlab; etc
- To simulate a random variable X , the main task is to design a procedure that takes $U \sim \text{Unif}(0, 1)$ as input and converts it into the random variable X



Basic Approach

- Example: Pseudocode to generate Bernoulli rv X with parameter p :

$U = \text{rand}; \leftarrow \text{Unif}(0, 1)$

if $U \leq p$

return $X = 1$;

else return $X = 0$

- To generate a rv X , we need
 - A primitive to generate $U \sim \text{Unif}(0, 1)$ (this is the only “random” part)
 - A procedure to convert U into X
 - This “procedure” is deterministic; can view it as a function/mapping

X takes values 0 or 1.

$$P(X=0) = P(U > p) = 1-p$$

because

$$\int_p^1 1 \, dx = 1-p$$

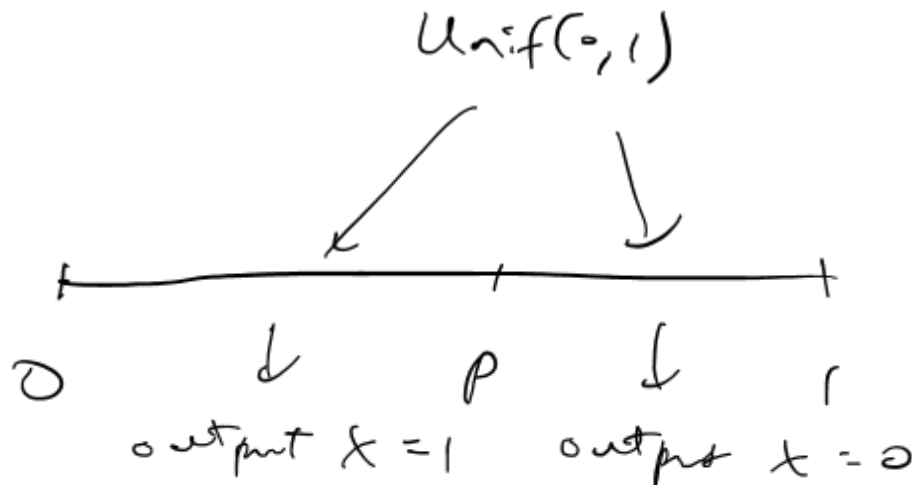
$$P(X=1) = P(U \leq p) = p$$

because $\int_0^p 1 \, dx = p$

$$\text{So } X \sim \text{Ber}(p)$$

Basic Approach

- Pseudocode to generate Bernoulli rv X with parameter p :
 $U = \text{rand};$
 if $U \leq p$
 return $X = 1;$
 else return $X = 0$
- Idea: divide the interval $[0, 1]$ up into two pieces $[0, p]$ and $[p, 1]$



Basic Approach

- Nothing special about X taking values 0 and 1
- Suppose X is a rv that has the pmf: $P(X=a) = p$; $P(X=b) = 1-p$
- Pseudocode to generate rv X :
 - $U = \text{rand};$
 - if $U \leq p$
 - return $X = a$;
 - else return $X = b$

X takes values a and b ,

$$P(X=a) = P(U \leq p) = p \quad \text{for } u \sim \text{unif}(0,1)$$

$$P(X=b) = P(U > p) = 1-p$$

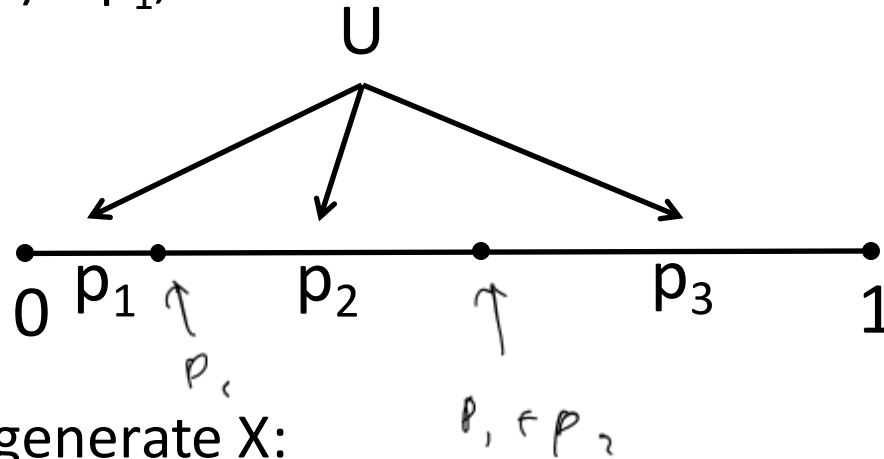
The output is correct.

Basic Approach

- Suppose we wish to generate a discrete random variable X taking three values x_1, x_2, x_3 , with pmf given by $p_1 = P(X = x_1)$, $p_2 = P(X = x_2)$ and $p_3 = P(X = x_3)$

Basic Approach

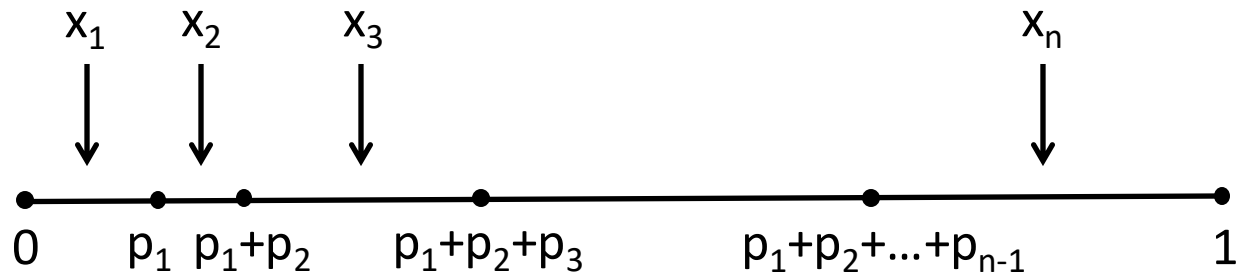
- Suppose we wish to generate a discrete random variable X taking three values x_1, x_2, x_3 , with pmf given by $p_1 = P(X = x_1)$, $p_2 = P(X = x_2)$ and $p_3 = P(X = x_3)$
- Can chop up the interval $[0, 1]$ into three pieces
- $P(U \text{ in first piece}) = p_1$, etc



- Pseudocode to generate X :
 $U = \text{rand};$
 if $0 \leq U \leq p_1$, return $X = x_1$;
 if $p_1 < U \leq p_1 + p_2$, return $X = x_2$;
 else return $X = x_3$

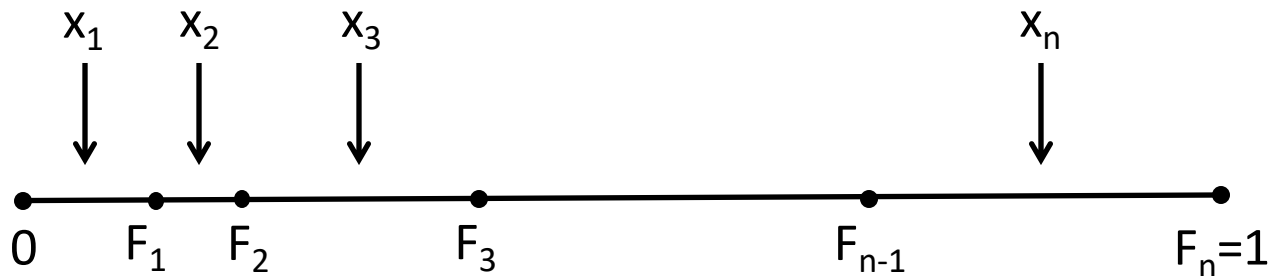
Basic Approach

- Can generalize the idea even further
- Suppose X is a general discrete random variable with pmf $p_k = P(X = x_k)$, $k = 1, 2, \dots, n$
- Suppose $x_1 < x_2 < \dots < x_n$
- Divide the interval $[0, 1]$ into n subintervals of lengths $p_1, p_2, \dots, p_n$ respectively
- Output $X = x_k$ if U lies within the k^{th} subinterval



Basic Approach

- More formally, define $F_k = p_1 + p_2 + \dots + p_k$, $k = 1, 2, \dots, n$
- F_k are the end points of the subintervals
- Output $X = x_k$ if U lies in (F_{k-1}, F_k) , where $F_0 = 0$

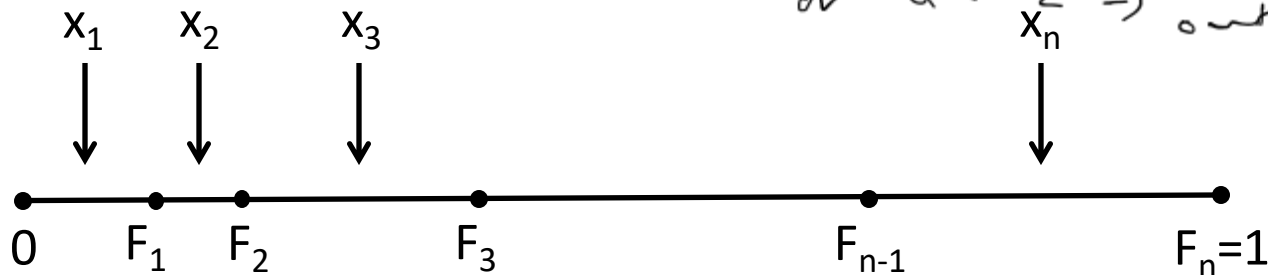


Basic Approach

- More formally, define $F_k = p_1 + p_2 + \dots + p_k$, $k = 1, 2, \dots, n$
- F_k are the end points of the subintervals
- Output $X = x_k$ if U lies in (F_{k-1}, F_k) , where $F_0 = 0$
- Pseudocode to generate X :

- $U \sim \text{Unif}(0, 1)$; Set $F_0 = 0$
- $k = 0$;
- while $U > F_k$
- $k = k+1$;
- output $X = x_k$

$F_0 = 0, k = 0$
1st iteration: $U > F_0 = 0 \Rightarrow k = 1$
2nd iteration: $U > F_1 \Rightarrow k = 2$
or $U \leq F_1 \Rightarrow \text{output } k = 1$
3rd iteration: $U > F_2 \Rightarrow k = 3$
or $U \leq F_2 \Rightarrow \text{output } k = 2$



This pseudo code is the same as

$$u \sim \text{unif}(0, 1)$$

if $F_{k-1} < u \leq F_k$, then output X_k .

The Inverse Transform Method

- The method applies the following function to $U \sim \text{Unif}(0, 1)$

- $$G(u) = \begin{cases} x_1 & \text{if } u \in [0, F_1] \\ x_2 & \text{if } u \in (F_1, F_2] \\ \dots & \\ x_n & \text{if } u \in (F_{n-1}, 1] \end{cases}$$

- What does the function G do?

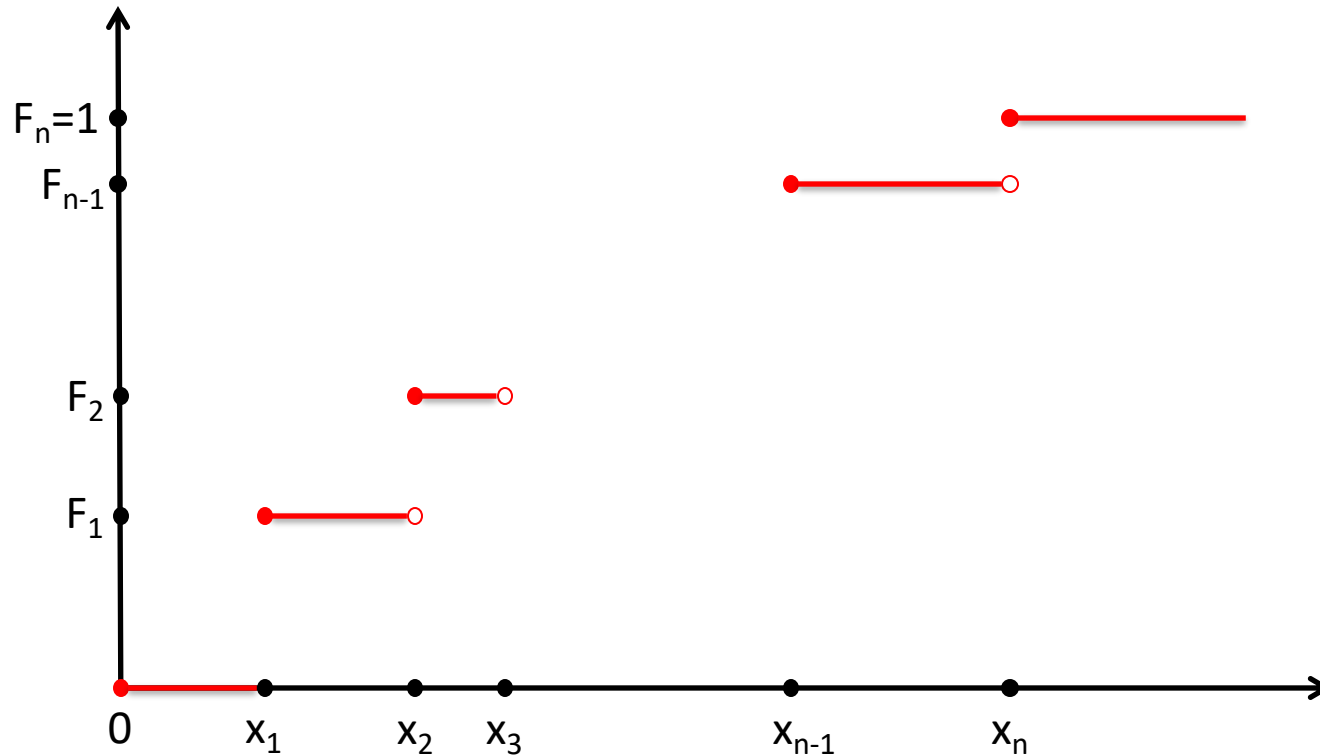
The pseudo code written before:

$u \sim \text{unif}(0, 1)$

output $G(u)$.

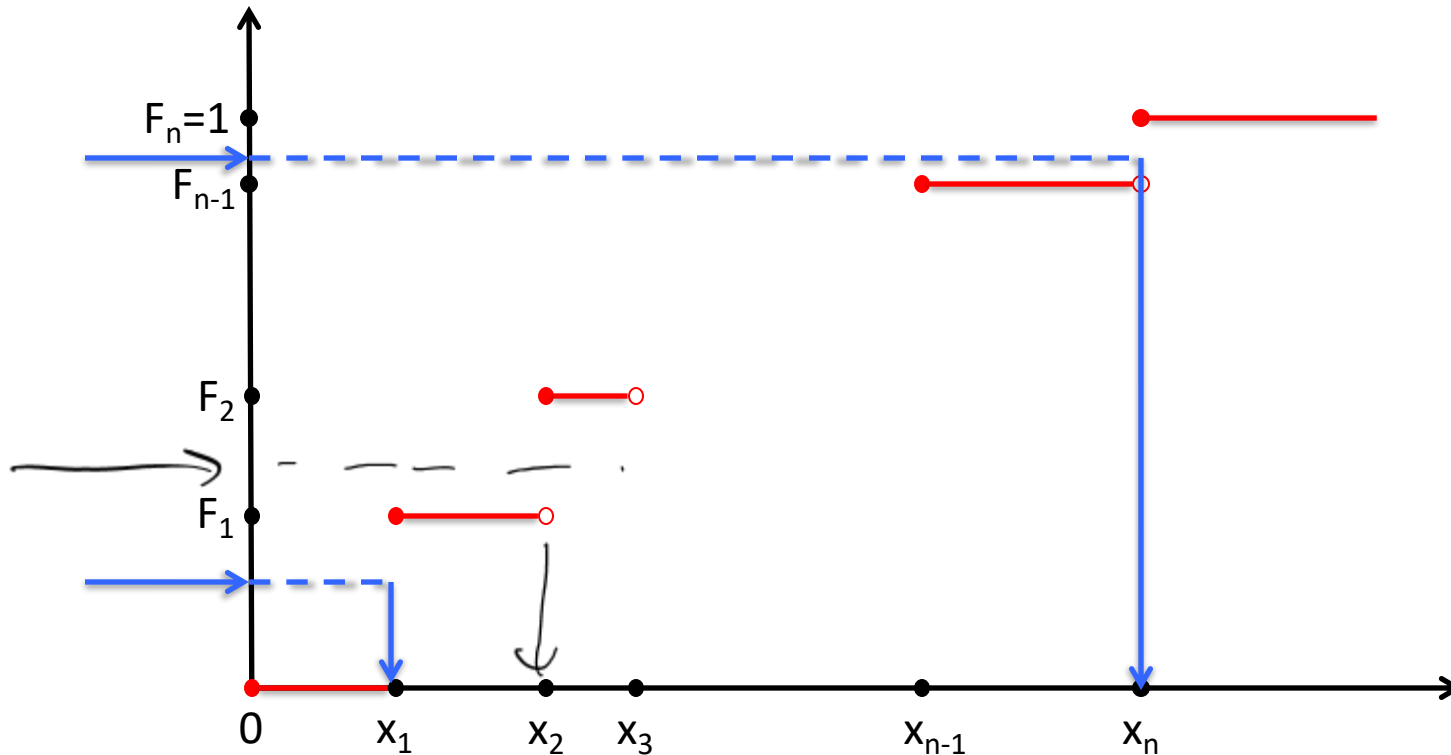
The Inverse Transform Method

- Plot of the cdf F looks like



The Inverse Transform Method

- This is what G does



- Hence the name “inverse transform”

Generating Binomial RV

- Let $X \sim \text{Bin}(n, p)$

Generating Binomial RV

- Let $X \sim \text{Bin}(n, p)$
- Calculate $p_k = \binom{n}{k} p^k (1-p)^{n-k}$ and $F_k = p_0 + p_1 + \dots + p_k$ for $k = 0, 1, \dots, n$, and $F_{-1} = 0$
- Alternately, use the recursion $F_{-1} = 0$ and $F_k = F_{k-1} + p_k$ for $k = 0, 1, 2, \dots, n$
- Pseudocode to generate X :
 - $U \sim \text{Unif}(0, 1)$; Set $F_{-1} = 0$
 - $k = -1$;
 - while $U > F_k$
 - $k = k+1$;
 - output $X = k$

$$k = -1, F_{-1} = 0$$

$$1^{\text{st}} \text{ iteration: } U > F_{-1} = 0$$

$$\Rightarrow k = 0$$

$$2^{\text{nd}} \text{ iteration: } U > F_0 = p_0$$

$$\text{if true} \Rightarrow k = 1$$

$$\text{if false} \Rightarrow \text{output } k = 0$$

⋮

Starting from $k = -1$ is because we
start the value of X from 0.

We can embed the recursion
into the loop:

$$U \sim \text{unif}, \quad F_{-1} = 0$$

$$k = -1$$

$$\text{while } U > F_k$$

$$k = k + 1$$

$$F_k = F_{k-1} + p_k$$

$$\text{output } X = k.$$

$$F_k = F_{k-1} + p_k$$

no need to
pre-compute

$$F_k, k=0, \dots, n.$$

Instead,

compute

F_k on the fly.

Alternative:

$$u \sim \text{unif}(0, 1), \quad F_0 = 0$$

$$k = 0$$

$$\text{while } u > F_k$$

$$k = k + 1$$

$$\text{Output } x_k = k - 1$$

Generating Binomial RV

- Sometimes we can just use the interpretation of the rv to simulate it or combine it with the inverse transform method
- Let $X \sim \text{Bin}(n, p)$; use Bernoulli trial interpretation

- Pseudocode:

```
- X = 0; I = 0;
```

```
for i=1:n
```

```
    U = rand;
```

```
    if U ≤ p
```

```
        I = 1;
```

```
    else I = 0;
```

```
    end
```

```
    X = X + I;
```

```
end
```

trace the output of $\text{Bin}(n, p)$

trace the success of Ber trial.

generate $\text{Ber}(p)$

- Which approach is preferable?

When p is small,

1st approach (TM on $\text{Bin}(n, p)$) is better because most likely we need few iterations in the while loop.

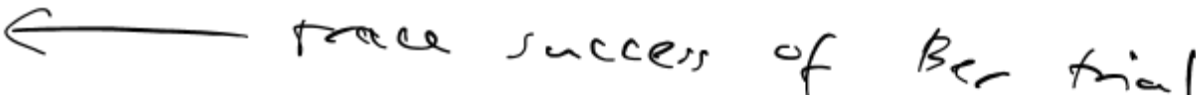
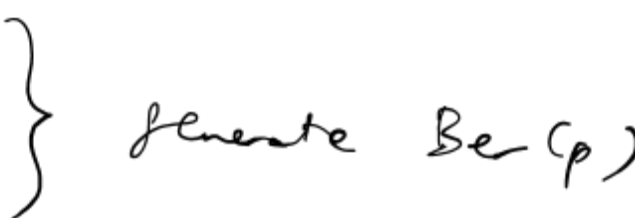
whereas 2nd approach (using Ber trials) still needs n iterations.

When p is large,

2nd approach is preferable.

Generating Geometric RV

- Let $X \sim \text{Geo}(p)$, with pmf $P(X = k) = (1-p)^{k-1}p$, $k = 1, 2, \dots$
- Using the interpretation of geometric rv, pseudocode to simulate a $\text{Geo}(p)$ distribution:

```
I = 0;  ← trace success of Ber trial  
X = 1;  ← trace the value of  $\text{Geo}(p)$   
while I = 0  
    U = rand;  
    if U ≤ p  
        I = 1;  } generate  $\text{Ber}(p)$   
    else X = X + 1;  
end  
end
```

Generating Geometric RV

- Use the inverse transform method

Generating Geometric RV

- Use the inverse transform method
 - $P(X = k) = p(1-p)^{k-1}, k = 1, 2, \dots$
 - $F_k = P(X = 1) + \dots + P(X = k) = 1 - P(X > k) = 1 - (1-p)^k$
 - Thus, generate $U \sim \text{Unif}(0, 1)$ and find k such that $F_{k-1} < U \leq F_k$
 - $X = \min\{k: U \leq F_k\} = \dots = \left\lceil \frac{\log(1-U)}{\log(1-p)} \right\rceil$
 - Simplify to get $\left\lceil \frac{\log(U)}{\log(1-p)} \right\rceil$

- Which approach is preferable?

$\lceil \cdot \rceil$ is "ceiling function"
smallest integers larger
or equal to
i.e., "round up".

For $\text{Geo}(p)$,

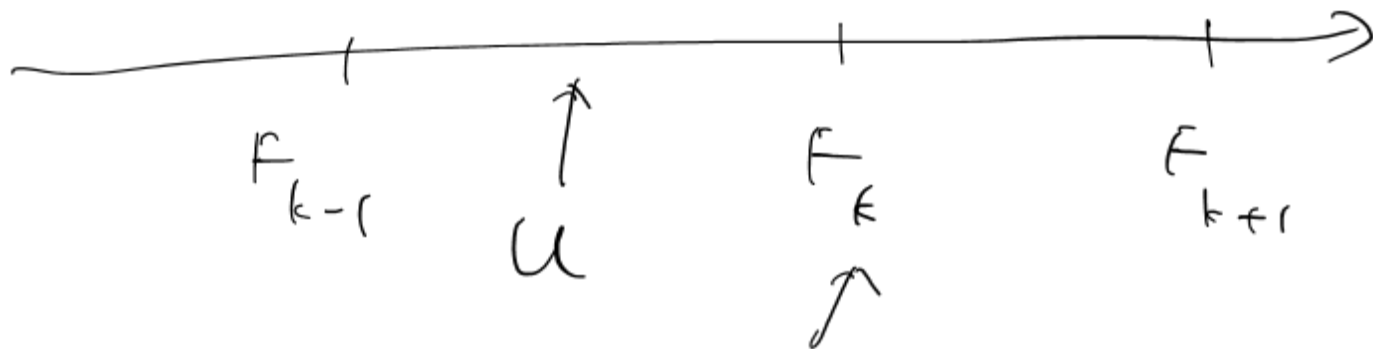
$$\begin{aligned} F_k &= P(X \leq k) = 1 - P(X > k) \\ &= 1 - (1-p)^k \end{aligned}$$

TM:

$$\text{if } F_{k-1} < u \leq F_k \Rightarrow \text{output } k.$$

This is the same as

$$\text{Output } \min \{ k : u \leq F_k \}$$



This k must be the smallest choice of k
such that $u \in F_k$.

To find $\min \{k: U \leq F_k\}$,

consider

$$U \leq F_k = 1 - (1-p)^k$$

$$\Rightarrow (1-p)^k \leq 1 - U$$

$$\Rightarrow k \log(1-p) \leq \log(1-u)$$

$$\Rightarrow k \geq \frac{\log(1-u)}{\log(1-p)} \Rightarrow \overset{\text{output}}{\left\lceil \frac{\log(1-u)}{\log(1-p)} \right\rceil}$$

When ρ is small,
2nd approach is better.

When ρ is large,
1st approach (using Ber trials) is better because
the iteration tends to stop after few iterations.
whereas 2nd approach (IRM) needs more complicated evaluation
of function.

Generating Poisson RV

- Let $X \sim \text{Poi}(\lambda)$
- pmf of X is $P(X = k) = p(k) = e^{-\lambda} \cdot \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$
- $F(k) = \sum_{i=0}^k p(k)$
- An alternate approach to compute $F(k)$:
 - $p(0) = e^{-\lambda}; F(0) = p(0)$
 - $p(k+1) = \frac{\lambda}{k+1} \cdot p(k)$ 
 - $F(k+1) = F(k) + p(k+1)$
- Apply the inverse transform method as before

because

$$\frac{p(k+1)}{p(k)} = \frac{\lambda}{k+1}$$

Generating Poisson RV

- Pseudocode

U = rand;

p = $\exp(-\lambda)$;

F = p;

i = 0;

while U > F

 p = $\lambda p / (i+1)$;

 F = F + p;

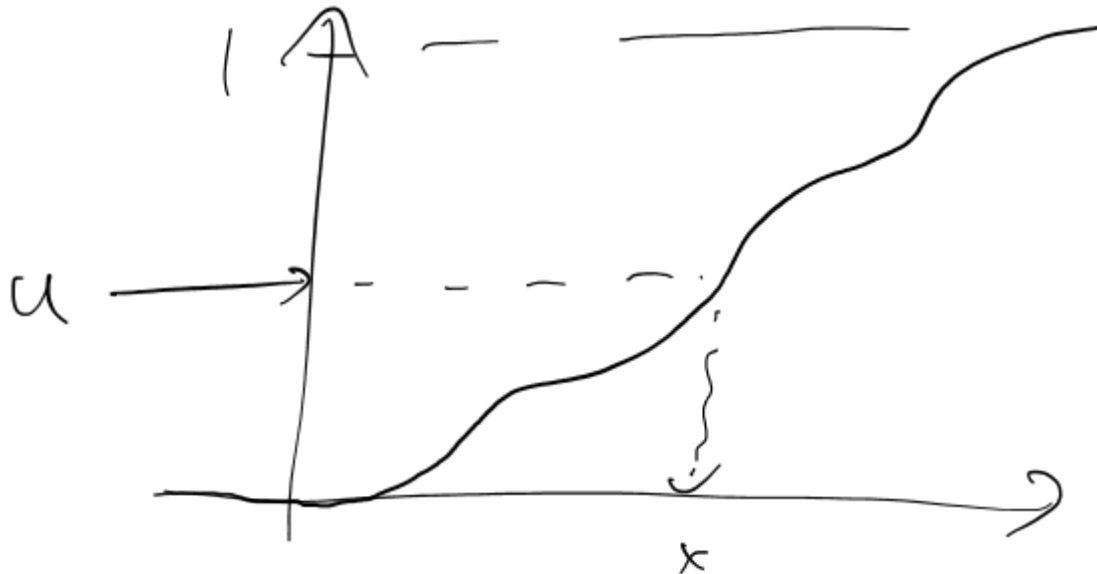
 i = i + 1;

end while

output i

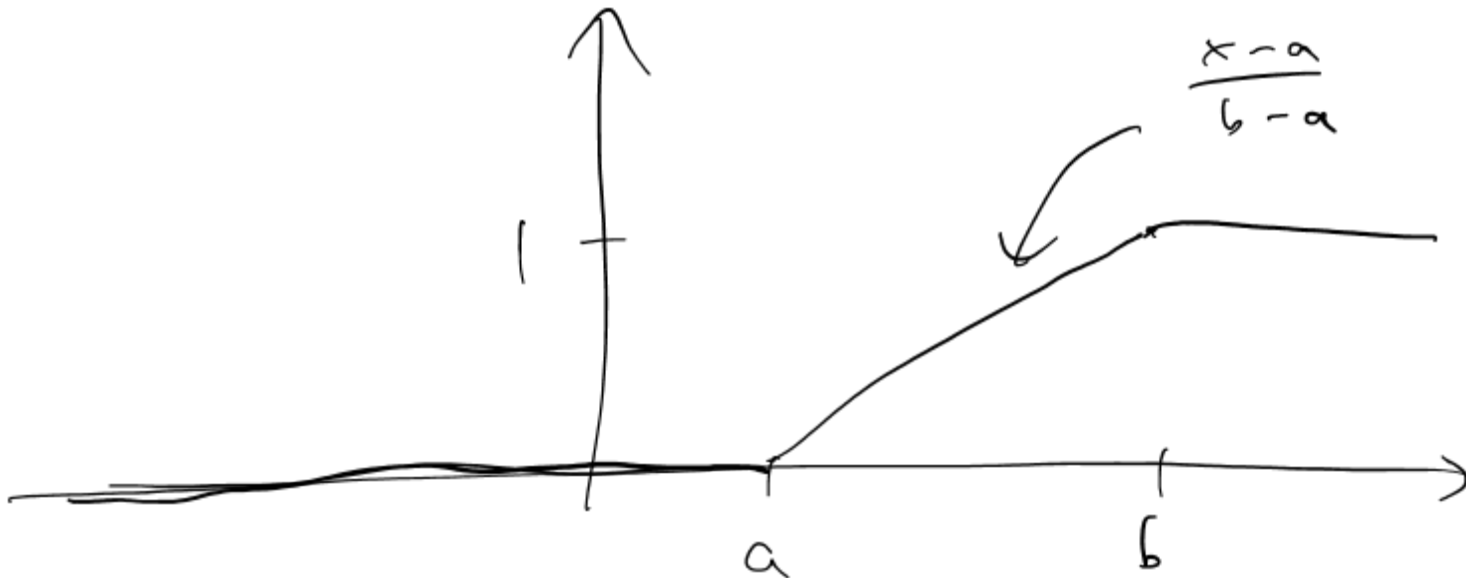
Inverse Transform Method for Continuous RV

- To generate a (continuous) random variable X with cdf $F(x)$:
 - Compute the inverse function $G(u)$
 - Generate $U \sim \text{Unif}(0, 1)$
 - Output $X = G(U)$



Generating Uniform RV

- Let $X \sim \text{Unif}(a, b)$
- If $a \leq x \leq b$, $P(X \leq x) = (x-a)/(b-a)$
- If $x < a$, $P(X \leq x) = 0$; if $x > b$, $P(X \leq x) = 1$



$$u = \frac{x - a}{b - a}$$

$$\Rightarrow x = a + (b - a)u$$

(TM for $\text{Unif}(a, b)$):

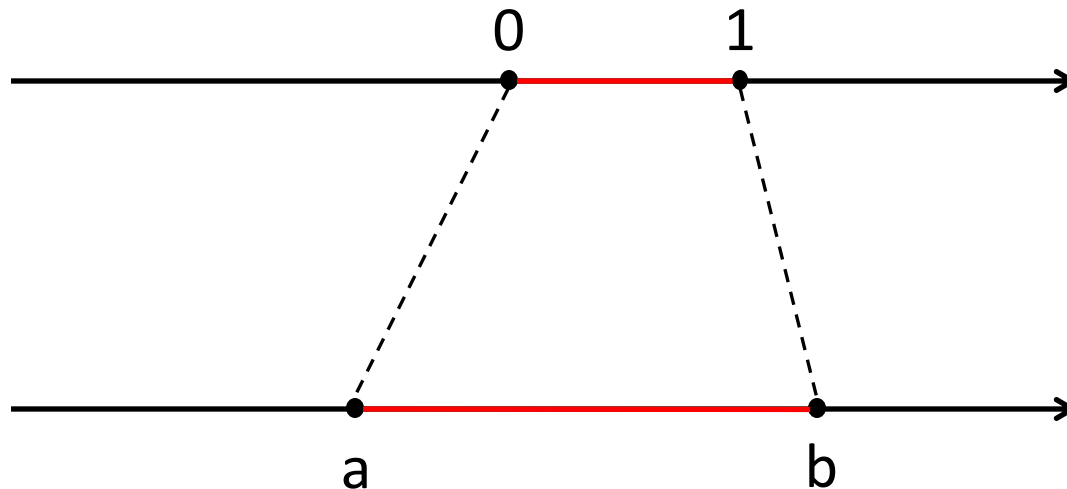
- Generate $u \sim \text{unif}(0, 1)$
- Output $a + (b - a)u$

Generating Uniform RV

- Let $X \sim \text{Unif}(a, b)$
- If $a \leq x \leq b$, $P(X \leq x) = (x-a)/(b-a)$
- If $x < a$, $P(X \leq x) = 0$; if $x > b$, $P(X \leq x) = 1$
- Thus $G(u) = (b-a)u + a$ for $0 < u < 1$
- Thus, to generate $\text{Unif}(a, b)$, we generate $U \sim \text{Unif}(0, 1)$ and output $X = (b-a)U + a$

Generating Uniform RV

- Let $X \sim \text{Unif}(a, b)$
- If $a \leq x \leq b$, $P(X \leq x) = (x-a)/(b-a)$
- If $x < a$, $P(X \leq x) = 0$; if $x > b$, $P(X \leq x) = 1$
- Thus $G(u) = (b-a)u + a$ for $0 < u < 1$
- Thus, to generate $\text{Unif}(a, b)$, we generate $U \sim \text{Unif}(0, 1)$ and output $X = (b-a)U + a$
- This matches the intuition that a linear transformation of a $\text{Unif}(0, 1)$ should also be uniform



Generating Exponential RV

- Let $X \sim \text{Exp}(\lambda)$
- The pdf $f(x)$ is given by

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{for } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

- The cdf $F(x)$ is given by

$$F(x) = 1 - e^{-\lambda x} \text{ for } x > 0 \text{ and } 0 \text{ otherwise}$$

Generating Exponential RV

- Let $X \sim \text{Exp}(\lambda)$
- The pdf $f(x)$ is given by

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{for } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

- The cdf $F(x)$ is given by

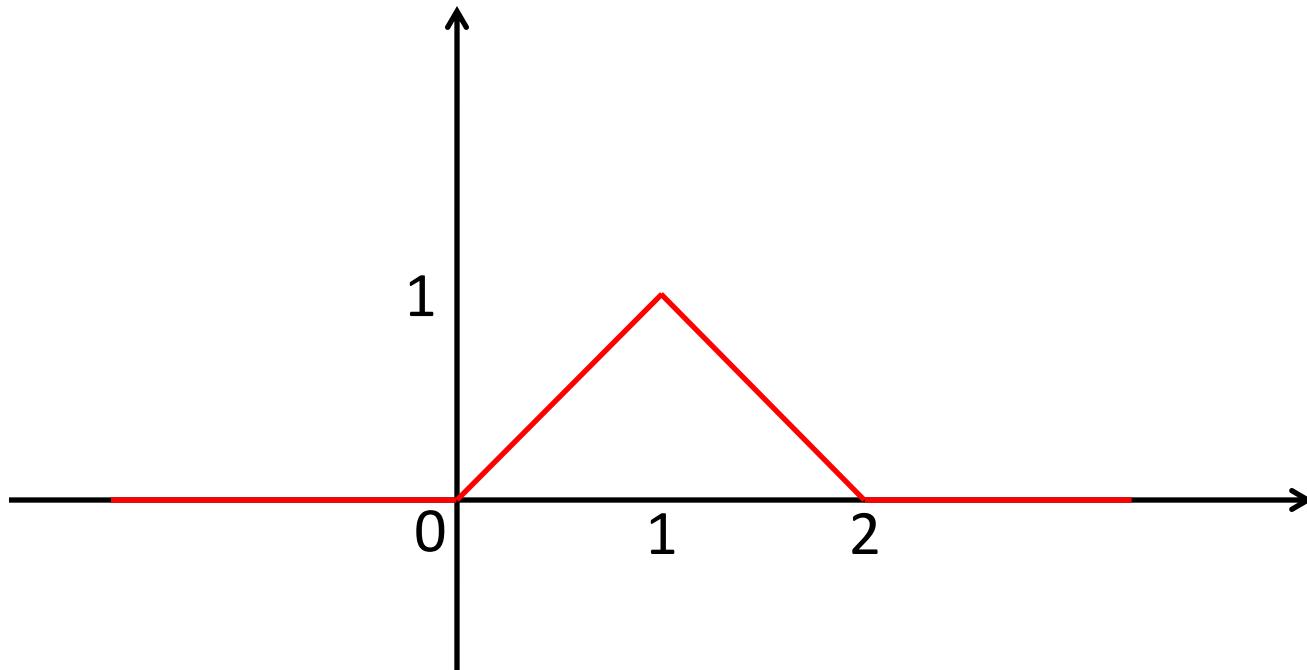
$$F(x) = 1 - e^{-\lambda x} \text{ for } x > 0 \text{ and } 0 \text{ otherwise}$$

- To use the inverse transform method, find the inverse function of F , which is $G(u) = -\log(1-u)/\lambda$
- Thus to generate X :
 - Generate $U \sim \text{Unif}(0, 1)$
 - Set $X = -\log(1-U)/\lambda$

Generating General Continuous RV

- **Example** (triangular density). Generate X with pdf given by

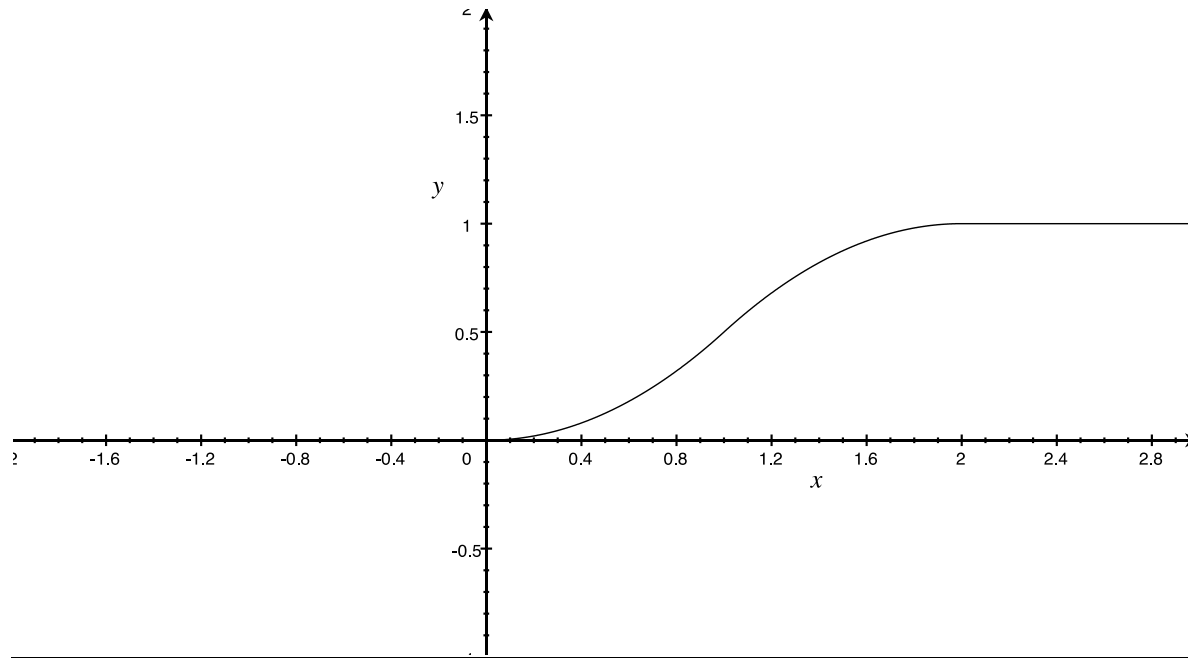
$$f(x) = \begin{cases} x & \text{if } x \in [0, 1] \\ 2 - x & \text{if } x \in [1, 2] \\ 0 & \text{otherwise} \end{cases}$$



Generating General Continuous RV

- To apply the inverse transform method, first compute the cdf $F(x)$

$$F(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ \frac{x^2}{2} & \text{if } x \in [0, 1] \\ -\frac{(2-x)^2}{2} + 1 & \text{if } x \in [1, 2] \\ 1 & \text{if } x \geq 2 \end{cases}$$



Generating General Continuous RV

- The inverse function is obtained as follows
- Set $u = x^2/2$; then $x = \sqrt{2u}$
- Set $u = -(2-x)^2/2 + 1$; then $x = 2 - \sqrt{2(1-u)}$
- Thus the inverse function $G(u)$ is

$$G(u) = \begin{cases} \sqrt{2u} & \text{if } u \in [0, 0.5]; \\ 2 - \sqrt{2(1-u)} & \text{if } u \in [0.5, 1] \end{cases}$$

- To generate X with the triangular density
 - Generate $U \sim \text{Unif}(0, 1)$
 - If $U < 0.5$, let $X = \sqrt{2U}$
 - If $U \geq 0.5$, let $X = 2 - \sqrt{2(1-u)}$

Limitations and Applications

- The inverse transform method applies as long as we can compute the cdf $F(x)$ (and its inverse)
- Not always possible to do; e.g., the normal density function
- For standard distributions, many software packages are available, but some probability distributions may not fit into any existing distributions, in which case the inverse transform method will be useful